

Encapsulation Overview

A stable Java/OOP reference for SHAYVERI CORE. This is not a build plan or an implementation checklist; it explains what code should be visible, what should be hidden, and why.

1. The rule in one sentence

Make a member private unless another class genuinely needs it as part of a deliberate contract. Encapsulation is not “make everything private.” It is exposing the smallest useful surface while hiding representation, mutable state, and implementation decisions.

A public API is a promise. Once another module depends on it, changing it becomes more expensive. Private code is free to change because nobody outside the class can depend on it.

2. Java visibility ladder

Modifier	Who can use it	CORE default use
private	Only this class	Fields, helper methods, internal state. This is the normal default.
(no modifier) <i>package-private</i>	Classes in the same package/module	Optional for module-internal helpers. Useful later; do not force it while learning.
protected	This class, package, and subclasses	Avoid by default. CORE does not need inheritance-heavy design.
public	Any code in the application (and consumers of a library)	Controllers, request records, cross-module interfaces, and intentional entry-point methods.

The practical beginner rule: use **public** for an intentional entry point; use **private** for everything that supports it. Reach for package-private only when you can name the module-only reason.

3. Records: A1 and A2

A request record is already strongly encapsulated. Its components are private and final; a public record exposes read-only accessor methods such as `placeId()`. There are no setters, so a caller cannot mutate the request after construction.

```
public record TelemetrySnapshotRequest(
    String placeId,
    Integer playerCount
) {}

// visible to callers: request.placeId()
// not possible: request.setPlaceId(...)
```

Do **not** try to write `private String placeId` inside the record body. The record header already declares the state. Your job in A1/A2 is validation and normalization, not ordinary field encapsulation.

4. Mutable Mongo documents: A3/A4

Mongo documents are normal classes because Spring Data needs to create and populate them when reading the database. Their fields should be private. The class offers only the construction and access paths the rest of the module actually needs.

```
@Document("telemetry_snapshots")
public class TelemetrySnapshot {
    @Id
    private String id;
    private String placeId;

    public TelemetrySnapshot() {} // Mongo reconstruction path
    public String getPlaceId() { return placeId; }
}
```

Keep fields private even when Mongo is involved. `@Id` and `@Document` describe persistence metadata; they do not require public fields. Only create setters where the document truly must be mutable. Prefer named lifecycle methods over a public setter that permits any state change.

5. Module 1 visibility map

Unit	Should be visible	Should be hidden
A1/A2 request records	The public record and its accessors: the HTTP/Roblox contract.	No mutable fields or setters. Validation/normalization details stay inside the record.
A3/A4 Mongo documents	Class, no-arg constructor for Mongo, intentional getters, factory method such as <code>from(...)</code> .	Fields, Mongo representation decisions, incidental setters.
A5 TelemetryStore	The interface and its method signatures: this is a cross-module seam.	How any implementation stores data.
A6 MongoTelemetryStore	Its Spring component identity and the TelemetryStore contract it implements.	Repository fields and persistence mechanics.
A7 TelemetryService	<code>accept(...)</code> and <code>acceptEvents(...)</code> : the module's application use cases.	Store/publisher/executor fields, conversion helpers, topic construction details.
A8 TelemetryController	HTTP endpoint methods - intentionally public framework entry points.	Injected service field and HTTP helper logic.
B1/B2 realtime	RealtimePublisher interface and <code>publish(topic, payload)</code> contract.	SimpMessagingTemplate and STOMP transport detail.
C1/C2 errors	ApiError record and global handler methods Spring calls.	Field-map construction and logging helpers.

6. Services, controllers, and dependency injection

A service has two separate surfaces: its *public use cases* and its *private machinery*. The controller needs to call `service.accept(request)`. It does not need access to the database store, WebSocket publisher, or executor used internally.

```

@Service
public class TelemetryService {
    private final TelemetryStore store;
    private final RealtimePublisher publisher;
    private final Executor executor;

    public void accept(TelemetrySnapshotRequest request) { ... }

    private String topicFor(String placeId) { ... }
}

```

Constructor injection is not a reason to make fields public. Spring calls the constructor once; after that the private final fields cannot be replaced. This protects the invariant that a service always has the dependencies it needs.

Controllers follow the same rule: endpoint methods are public because Spring calls them; the injected service field is private final. A controller should not expose persistence or security details as its API.

7. Interfaces and adapters: deliberately public contracts

The store rule introduced for a future asdb backend is an encapsulation decision. A service depends on `TelemetryStore`, not `MongoTelemetryStore`. That interface must be visible where the service needs it. The Mongo adapter's internal repositories stay hidden.

```

public interface TelemetryStore {
    void saveSnapshot(TelemetrySnapshot snapshot);
}

@Component
public class MongoTelemetryStore implements TelemetryStore {
    private final TelemetrySnapshotRepository snapshots;
    // Mongo-specific implementation stays here
}

```

This is dependency inversion in concrete terms: change the adapter later, not every caller. Do not make an adapter's database client or repository public “just in case.”

8. Collections, maps, and leaking mutable state

A private field is not fully encapsulated if a public getter hands out the same mutable collection. The caller could change the object indirectly.

```

// Avoid: caller can modify internal state
public Map getMetrics() { return metrics; }

// Prefer one of these:
public Map getMetrics() { return Map.copyOf(metrics); }
// or store an immutable map from construction onward.

```

Request records normalize optional maps to `Map.of()` for the same reason: downstream code sees a stable, non-null value rather than a mutable or absent implementation detail.

9. A quick decision checklist

Ask this	If yes	Default
Must another module or an HTTP client call it?	It is a deliberate contract.	public
Does only this class need it to do its job?	It is implementation detail.	private
Do several classes in this one module need it, but no other module should?	It is module-internal.	package-private can fit
Would exposing it let callers put the object into an invalid state?	Protect an invariant.	private field; named method or no mutator
Is it a record component?	Records generate safe read accessors already.	keep it in the header

The goal is not maximum secrecy. It is code that can be understood and changed locally: callers know *what* they may ask a unit to do, while the unit retains control of *how* it does it.

10. Stable default for this project

For SHAYVERI CORE, use this starting policy: **private fields; private helpers; public request records; public controllers and endpoint methods; public service use-case methods; public store interfaces; private adapter internals.** When an exception appears, document the reason in the unit's dedicated documentation after the design is settled.

This avoids both bad extremes: a bag of public fields that any code can mutate, and a system so hidden that legitimate modules cannot communicate. Each module gets a small, intentional public face and owns everything behind it.